Predict Returns with OLS, Then Compare Stockholding Decisions

Use ordinary least squares (OLS) to forecast the price return over five holding intervals, then examine how thresholds, amounts invested and exits change the account.

Issues	Reading focus
1-4	Identify the forecast period and when its outcome becomes known
5-9	How the line is estimated: derivation, numbers and code checks
10-14	2023 model selection; six 2024 forecasts and errors
15-21	Stock, cash, allocation and early exits under the same forecasts
22-24	All 24 periods, fees charged and stock purchase amounts
25	What can the model say when current input exceeds its training range?

A forecast is a return percentage, not a stock price, probability of a rise or profit guarantee. Tables, figures and calculations use the same AAPL data. Trading rules and execution are classroom assumptions.

AAPL Closes: Start with Specific Dates and Amounts

Use 2023 for estimation and validation, and 2024 for subsequent comparisons without revising the model retrospectively.

Before running: Decide on 1/2 and buy at the 1/3 close. Which price determines the shares purchased?

```
import numpy as np
import pandas as pd
import yfinance as yf
aapl = yf.Ticker("AAPL").history(start="2023-01-01",
    end="2024-07-01", interval="1d", auto_adjust=False, actions=True)
aapl.index = aapl.index.strftime("%Y-%m-%d")
close = aapl["Close"]
assert close.index.is_unique and close.index.is_monotonic_increasing
assert close.gt(0).all() and aapl["Stock Splits"].eq(0).all()
dates = ["2024-01-02", "2024-01-03", "2024-01-10"]
pd.DataFrame({"Observation date": dates, "Close (USD/share)": close.loc[dates].round(2).values})
```

Observation date	Close (USD/share)
2024-01-02	185.64
2024-01-03	184.25
2024-01-10	186.19

There are 374 price observations. This part counts only Close price changes, excluding dividends. The purchase price here is 184.25 USD/share on 1/3, not 185.64 on 1/2.

Account	Shared conditions this part
Initial capital	1,000 USD cash; no additions or borrowing; fractional shares allowed
Adjustment	Execute at the next close after deciding; decide every fifth trading observation
Continue holding	Retain shares while stock is still required; do not sell and repurchase at the same price
Comparison	24 complete holding periods; common 6/26 valuation, without forced sale
Fees	No fees initially; later examples charge 1 USD per buy/sell; no taxes or interest

ISSUE 01

Which return after purchase are we forecasting?

Decide on 1/2 and buy at the 1/3 close: forecast the return over the five price intervals after purchase.

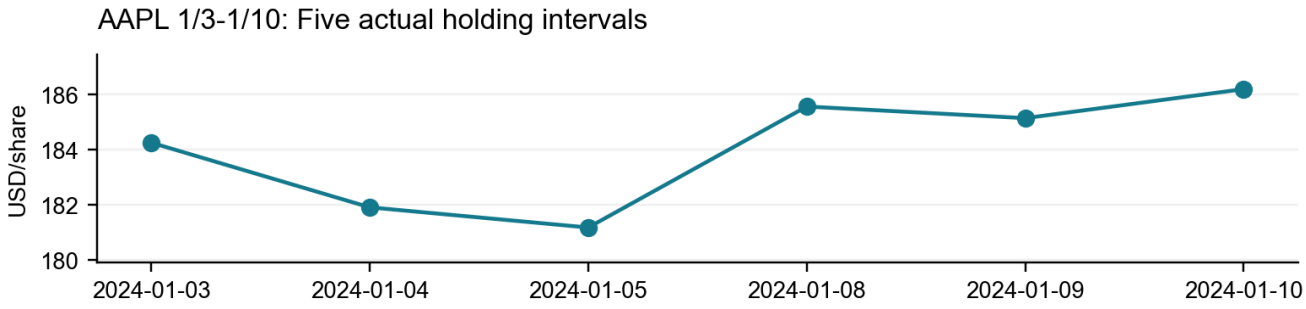
The five intervals are 1/3→4, 4→5, 5→8, 8→9 and 9→10, not five calendar days. The 1/2→3 change precedes purchase and is not holding profit/loss. Next-close execution is specified for this example; it does not mean US stocks cannot trade after hours.

Before running: How many intervals connect six closes? Should the starting price be 185.64 or 184.25?

```
held_dates = close.loc["2024-01-03":"2024-01-10"].index
held_price = close.loc[held_dates]
first_answer = held_price.iloc[-1]/held_price.iloc[0]-1
pd.DataFrame({"Observation date": held_dates,
    "Close (USD/share)": held_price.round(2).values})
```

Observation date	Close (USD/share)
2024-01-03	184.25
2024-01-04	181.91
2024-01-05	181.18
2024-01-08	185.56
2024-01-09	185.14
2024-01-10	186.19

$$\frac{186.19}{184.25} - 1 \approx 1.0529\%$$



Buy at 184.25 on 1/3 and value at 186.19 on 1/10. The price return is about 1.0529%, unknown when deciding on 1/2.

Discussion

If you mistakenly calculate from 185.64 on 1/2 to 186.19 on 1/10, is that still the return being forecast here?

Sources: [1], [7].

ISSUE 02

How do the past five price intervals become a model input?

Use the past five intervals as input. They differ from the five intervals after purchase that provide the outcome.

C is Close (USD/share), and t indexes trading observations. x is the past five-interval return ending at t; y runs from the next close t+1 to t+6, also five intervals. Both x and y initially use decimal returns; tables multiply by 100 to display percentages.

$$x_t = \frac{C_t}{C_{t-5}} - 1, \qquad y_t = \frac{C_{t+6}}{C_{t+1}} - 1$$

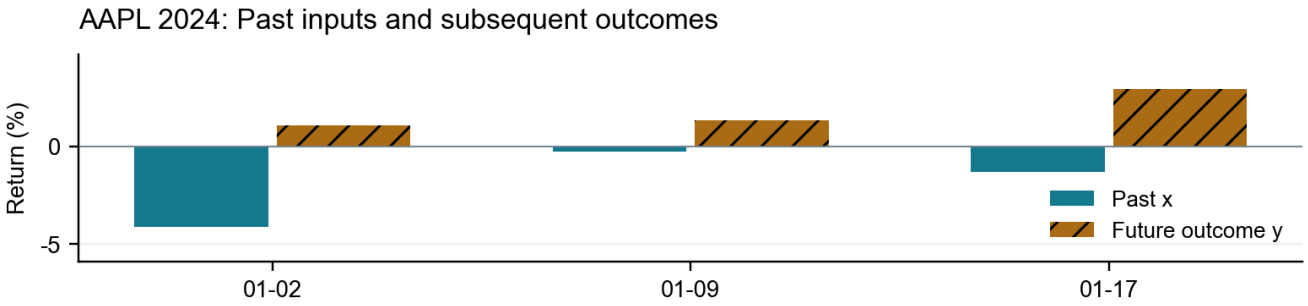
For 1/2, x uses 12/22 and 1/2, while y uses 1/3 and 1/10. The two periods share no price endpoint.

$$x_{1/2} \approx \frac{185.64}{193.60} - 1 \approx -4.1116\%$$

Before running: The past return on 1/2 is a decline of 4.1116%. Must the return after purchase also be negative?

```
x = close/close.shift(5)-1
y = close.shift(-6)/close.shift(-1)-1
observation_dates = pd.Series(close.index, index=close.index)
model = pd.DataFrame({"x": x, "y": y,
    "Entry": observation_dates.shift(-1), "End": observation_dates.shift(-6)})
dates = ["2024-01-02", "2024-01-09", "2024-01-17"]
pd.DataFrame({"Decision date": dates,
    "Past five intervals (%)": (100*x.loc[dates]).round(4).values,
    "Adjustment date": model.loc[dates, "Entry"].values,
    "Outcome date": model.loc[dates, "End"].values,
    "Outcome (%)": (100*y.loc[dates]).round(4).values})
```

Decision date	Past five intervals (%)	Adjustment date	Outcome date	Outcome (%)
2024-01-02	-4.1116	2024-01-03	2024-01-10	1.0529
2024-01-09	-0.2693	2024-01-10	2024-01-18	1.3105
2024-01-17	-1.3287	2024-01-18	2024-01-25	2.937



The first three x values are negative, but the subsequent five-interval outcomes are positive. The columns cover different periods, not two ways of calculating the same return.

Discussion

For 1/9, the input uses 1/2 and 1/9, while the outcome uses 1/10 and 1/18. Which period can serve as input at the 1/9 close?

Sources: [1], [7].

ISSUE 03

Which outcomes are known on the decision date?

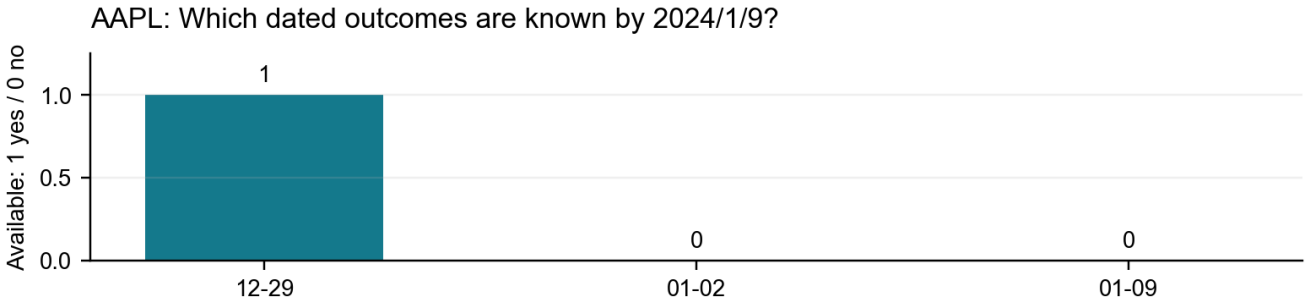
A known input does not imply a known complete outcome. Eligibility for training depends on the outcome's endpoint.

Entry is the adjustment date here; End is the outcome date for the five-interval price return. At the 1/9 close, when preparing the second decision, the first period 1/3→1/10 is unfinished. The outcome for the dated observation on 1/2 cannot yet update the model.

Before running: At the 1/9 close, are the outcomes for both dated observations on 12/29 and 1/2 known?

```
known = model.dropna(subset=["x", "y"])
asof = "2024-01-09"
available = known.loc[known["End"] <= asof]
dates = ["2023-12-29", "2024-01-02", "2024-01-09"]
pd.DataFrame({"Input date": dates, "Outcome date": model.loc[dates, "End"].values,
    "Known at 1/9 close": (model.loc[dates, "End"] <= asof).map({True: "Yes", False: "No"}).values})
```

Input date	Outcome date	Known at 1/9 close
2023-12-29	2024-01-09	Yes
2024-01-02	2024-01-10	No
2024-01-09	2024-01-18	No



The 12/29 observation obtains its full outcome on 1/9 and is usable that evening. The 1/2 observation must wait until 1/10, and 1/9 until 1/18; the latter two are not yet usable.

Calculating 363 outcomes afterward does not make them all known when forecasting. This issue checks availability. The later fixed model uses only the 2023 training period and is not updated on 1/9. Do not fill unknown outcomes with 0.

Discussion

If the model is refitted at the 1/9 close using the 1/2 observation, which not-yet-known Close is used prematurely?

Sources: [4], [5], [7].

ISSUE 04

How are training, validation and 2024 evaluation separated?

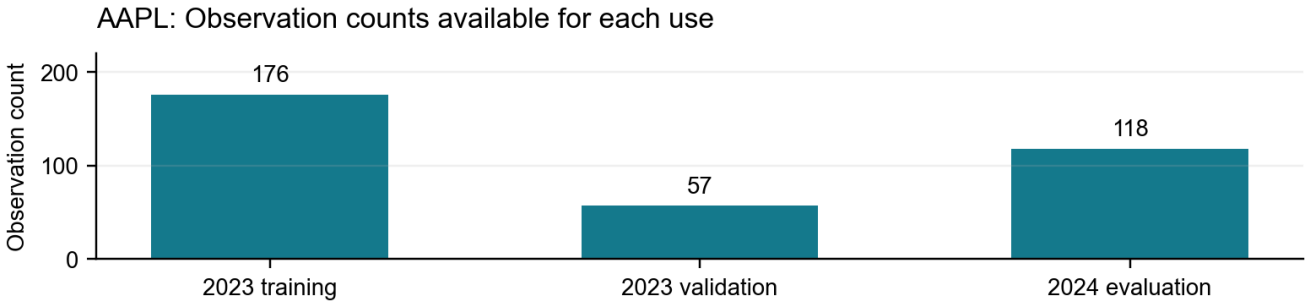
Training estimates coefficients; validation chooses the setting; 2024 evaluates the locked model. Outcomes must not extend past each period's cutoff.

Before running: The outcome for the 9/22 observation ends on 10/2. Can it enter training ending on 9/29?

```
train = known.loc[(known.index >= "2023-01-03") &
    (known["End"] <= "2023-09-29")]
valid = known.loc[(known.index >= "2023-10-02") &
    (known["End"] <= "2023-12-29")]
test = known.loc[known.index >= "2024-01-02"]
valid_cycles, test_cycles = valid.iloc[::5].copy(), test.iloc[::5].copy()
parts = {"Training": train, "Validation": valid, "2024 evaluation": test}
pd.DataFrame([[name, len(rows), rows.index[0], rows["End"].iloc[-1]]
    for name, rows in parts.items()],
    columns=["Use", "Observation count", "First input", "Last outcome"])
```

Use	Observation count	First input	Last outcome
Training	176	2023-01-10	2023-09-29
Validation	57	2023-10-02	2023-12-29
2024 evaluation	118	2024-01-02	2024-06-28

The last training input date is 9/21; its outcome completes on 9/29. The six observations in each of 9/22-29 and 12/21-29 have outcomes spanning the split boundaries, so the fixed split excludes them. Their outcomes are not changed to 0.



A daily observation need not be an independent trade. Take every fifth observation for 12 validation periods; 2024 similarly gives 24 evaluation periods, with the last complete outcome on 6/26.

For two adjacent dates, the five-interval outcomes share four price changes. For example, the 1/2 outcome uses 1/3 to 10, and the 1/3 outcome uses 1/4 to 11. They are usable in training but are not 176 independent experiences. The account decides every fifth observation; nonoverlapping intervals need not be independent either.

Discussion

If you change the model choice after seeing 2024 results, can the same 2024 data still be called evaluation data unused for selection?

Sources: [2], [4], [5].

ISSUE 05

How does a line predict, and what is one prediction error?

A prediction is intercept plus slope times input. The error is the subsequently observed outcome minus the prediction.

b_0 is the intercept (decimal return), b_1 the slope (unitless), and $\hat{y}$ the return prediction. e=y-$\hat{y}$ also uses decimal returns; the table displays 100e in percentage points. First try a horizontal line with b_1=0 and b_0=mean training outcome. Estimate the slope later rather than assuming the final OLS coefficients now.

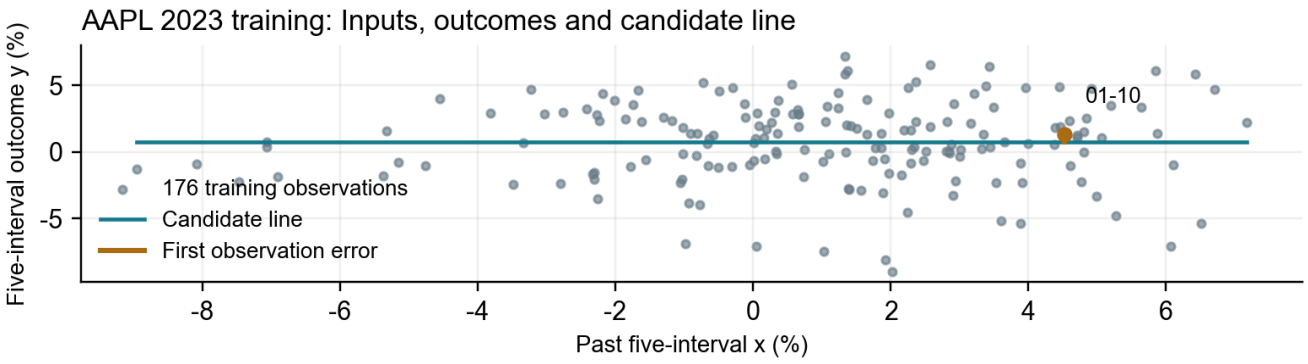
$$\hat{y} = b_0 + b_1 x, \qquad e = y - \hat{y}$$

Before running: Is this outcome above the training mean? Should actual minus predicted be positive or negative?

```
xbar, ybar = train[["x", "y"]].mean()
trial_prediction = pd.Series(ybar, index=train.index)  # Temporarily set slope to 0
trial_errors = train["y"]-trial_prediction
first = train.iloc[0]
pd.DataFrame({"Decision date": [train.index[0]],
    "x (%)": [round(100*first.x, 4)], "Outcome (%)": [round(100*first.y, 4)],
    "Prediction (%)": [round(100*ybar, 4)],
    "Error (percentage points)": [round(100*trial_errors.iloc[0], 4)]})
```

Decision date	x (%)	Outcome (%)	Prediction (%)	Error (percentage points)
2023-01-10	4.5255	1.3334	0.7472	0.5862

$$e \approx 0.01333432 - 0.00747204 \approx 0.00586228$$



For the 1/10 observation, outcome 1.3334% minus prediction 0.7472% gives an error of about 0.5862 percentage points. The vertical line shows that difference, not stock profit/loss.

Discussion

With slope temporarily set to 0, x on 1/11 differs from 1/10. Does this candidate line's prediction change?

Sources: [2], [3].

ISSUE 06

Why choose a line by the sum of squared errors?

Positive and negative errors can cancel. Squaring makes every deviation add to the sum being minimized.

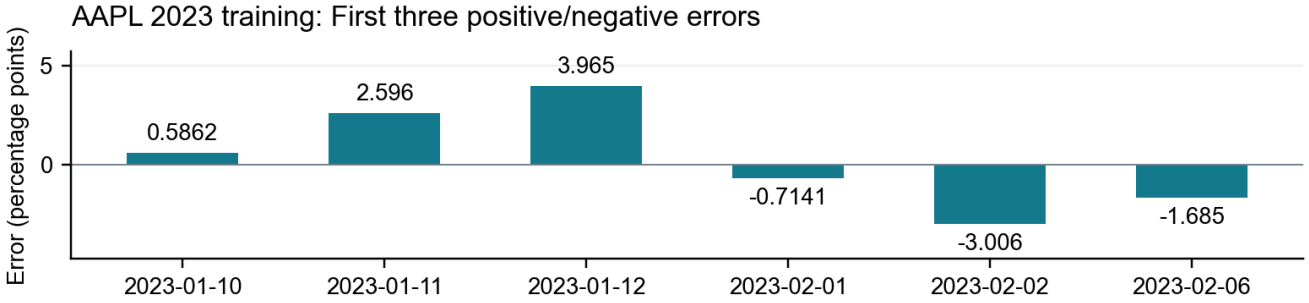
e remains decimal-return error; i indexes training observations. RSS is the residual sum of squares, summing squared errors over 176 observations. It is not USD profit/loss or a claim that every prediction is correct. OLS finds b_0, b_1 minimizing RSS.

$$\mathrm{RSS} = \sum_{i=1}^{176} e_i^2 = \sum_{i=1}^{176} (y_i - b_0 - b_1 x_i)^2$$

Before running: Using the training mean, the errors sum to nearly 0. Must their squared sum also be nearly 0?

```
trial_rss = (trial_errors**2).sum()
shown_errors = pd.concat([trial_errors[trial_errors>0].head(3),
    trial_errors[trial_errors<0].head(3)]).sort_index()
pd.DataFrame({"Error sum, 176 observations": [f"{trial_errors.sum():.10f}"],
    "Squared sum, 176 observations": [round(trial_rss, 8)],
    "Positive error count": [int((trial_errors>0).sum())],
    "Negative error count": [int((trial_errors<0).sum())]})
```

Error sum, 176 observations	Squared sum, 176 observations	Positive error count	Negative error count
0.0000000000	0.1621232	91	85



Only six errors are displayed to show their signs; they do not replace all 176 observations. The full error sum is near 0 but the squared sum is 0.16212320 (squared decimal returns). Predictions have errors; they are not all correct.

The first error is about 0.00586228, with square about 0.0000343663; a negative error also has a positive square. Squaring gives greater weight to larger deviations but does not guarantee the greatest trading profit.

Discussion

If an error doubles in magnitude, by what factor does its RSS contribution change? Explain using 0.00586228 on this page.

Sources: [2].

ISSUE 07

For a given slope, why do two means determine the intercept?

Fix the slope first, then select the intercept minimizing the sum of squared errors over 176 observations. The intercept is not a stock price or rise probability.

Bars over x and y denote means of the 176 training observations, still using decimal returns. Each $e=y-b_0-b_1x$ has derivative -1 with respect to b_0, so e^2 has derivative -2e. Summing gives:

$$\frac{\partial \mathrm{RSS}}{\partial b_0} = -2\sum e_i = 0$$

$$\sum y_i - 176b_0 - b_1\sum x_i = 0 \quad \Rightarrow \quad b_0 = \bar{y} - b_1\bar{x}$$

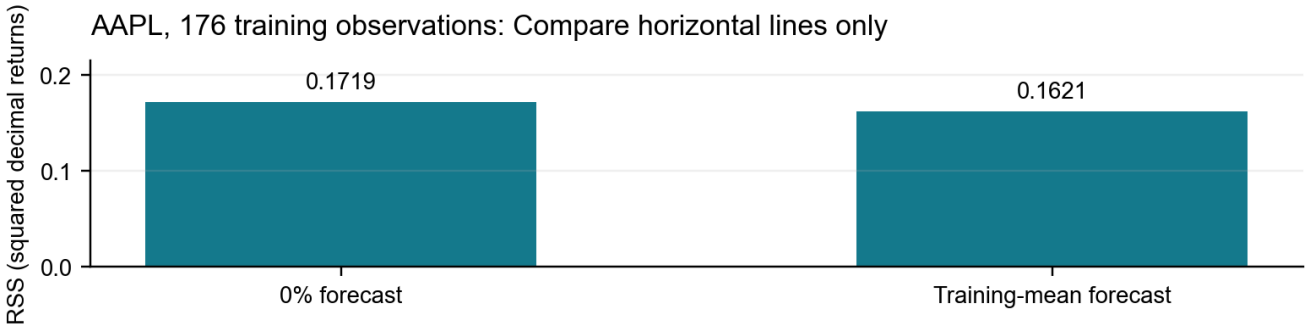
Divide the sum equation by 176 to obtain the mean equation. With slope fixed, RSS is an upward-opening quadratic in the intercept, so a derivative of 0 identifies its minimum.

Before running: Which column equals Σy divided by 176? What is the intercept if the slope is 0?

```
n = len(train)
sum_x, sum_y = train["x"].sum(), train["y"].sum()
zero_rss = (train["y"]**2).sum()
pd.DataFrame({"Observation count": [n], "Mean x": [round(xbar, 8)], "Mean y": [round(ybar, 8)],
    "Σx": [round(sum_x, 8)], "Σy": [round(sum_y, 8)]})
```

Observation count	Mean x	Mean y	Σx	Σy
176	0.00978299	0.00747204	1.7218056	1.31507939

$$b_1 = 0: \quad b_0 = \bar{y} \approx 0.00747204 = 0.7472\%$$



With both slopes 0, predicting the mean gives RSS 0.16212320, below 0.17194953 for predicting 0. This is training fit, not 2024 trading profit.

Discussion

For discussion only, set the slope temporarily to 0.1. Using mean x of 0.00978299 and mean y of 0.00747204, what is the intercept?

Sources: [2].

ISSUE 08

How are the slope's numerator and denominator found?

Subtract means, then sum products and squares; one observation cannot determine the line.

The derivative of e with respect to slope is -x, so the RSS derivative is -2Σxe; set it to 0. Subtract mean x times Σe=0 to obtain Σ(x-mean x)e=0. Substituting the previous intercept equation gives e=(y-mean y)-b_1(x-mean x). Rearranging:

$$\sum(x_i - \bar{x})(y_i - \bar{y}) = b_1 \sum(x_i - \bar{x})^2$$

$$b_1 = \frac{\sum(x_i - \bar{x})(y_i - \bar{y})}{\sum(x_i - \bar{x})^2}, \qquad b_0 = \bar{y} - b_1 \bar{x}$$

Mean x≈0.00978299 and mean y≈0.00747204. The x values differ, so the denominator>0. Numerator and denominator have the same units, making the slope unitless.

Before running: Does the slope use the first observation ratio or the ratio of sums over all 176 observations?

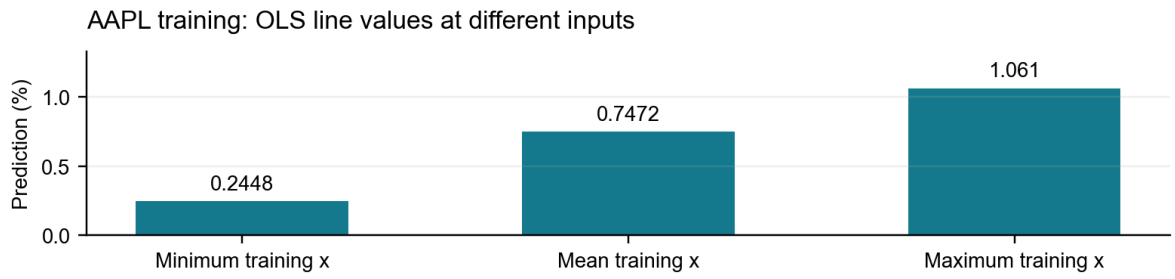
```
dx, dy = train["x"]-xbar, train["y"]-ybar
numerator, denominator = (dx*dy).sum(), (dx**2).sum()
assert denominator > 0
b1_hand = numerator/denominator
b0_hand = ybar-b1_hand*xbar
pd.DataFrame({"Observations included": ["First observation, 1/10", "Sum over 176 observations"],
    "(x-mean x)(y-mean y)": [(dx*dy).iloc[0], numerator],
    "(x-mean x)²": [(dx**2).iloc[0], denominator]}).round(8)
```

Observations included	(x-mean x)(y-mean y)	(x-mean x)²
First observation, 1/10	0.00020794	0.00125824
Sum over 176 observations	0.00840671	0.1662779

First x≈0.04525463, y≈0.01333432. Subtract their means: about 0.03547164, 0.00586228. Their product is the first table entry.

$$b_1 \approx \frac{0.0084067079}{0.1662778959} \approx 0.05055818$$

$$b_0 \approx 0.00747204 - 0.05055818(0.00978299) \approx 0.00697743$$



Substituting the minimum and maximum training x into the line gives predictions of about 0.2448% to 1.0610%. These are model values, not observed returns.

Discussion

On this line, how many percentage points does the prediction increase when the input increases by 1 percentage point? First convert 1 percentage point to 0.01.

Sources: [2], [3].

ISSUE 09

Do the code and sum formulas give the same line?

With the same 176 training observations and line including an intercept, the two calculations check one OLS problem. They do not compare two forecasting methods.

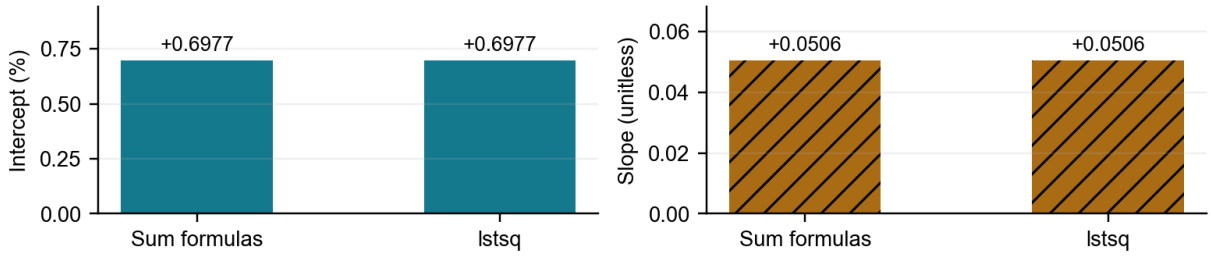
design arranges the 176 training observations in two columns: all 1 values for the intercept, and x values for the slope. lstsq minimizes the residual sum of squares; its coefficients should match the preceding sum formulas.

Before running: With the same data and objective, should the two coefficient columns agree to eight decimal places?

```
design = np.column_stack([np.ones(len(train)), train["x"]])
b0, b1 = np.linalg.lstsq(design, train["y"], rcond=None)[0]
fit_errors = train["y"]-(b0+b1*train["x"])
pd.DataFrame({"Coefficient": ["b0", "b1"], "Sum formulas": [b0_hand, b1_hand],
    "lstsq": [b0, b1]}).round(8)
```

Coefficient	Sum formulas	lstsq
b0	0.00697743	0.00697743
b1	0.05055818	0.05055818

AAPL, 176 observations: The same coefficients from two calculations



Each pair matches: intercept 0.697743% and slope 0.05055818. Their units differ; do not compare bar heights across the two sides.

The absolute values of Σe and Σxe are both below 0.000000000001, satisfying the two derivative conditions. This does not make every prediction correct. The coefficients use training only, excluding validation and 2024 outcomes.

Discussion

If you remove design's column filled with 1, can it still predict the return of about 0.697743% shown here at x=0?

Sources: [2], [3].

ISSUE 10

How does a line fitted to 176 or 80 training observations differ?

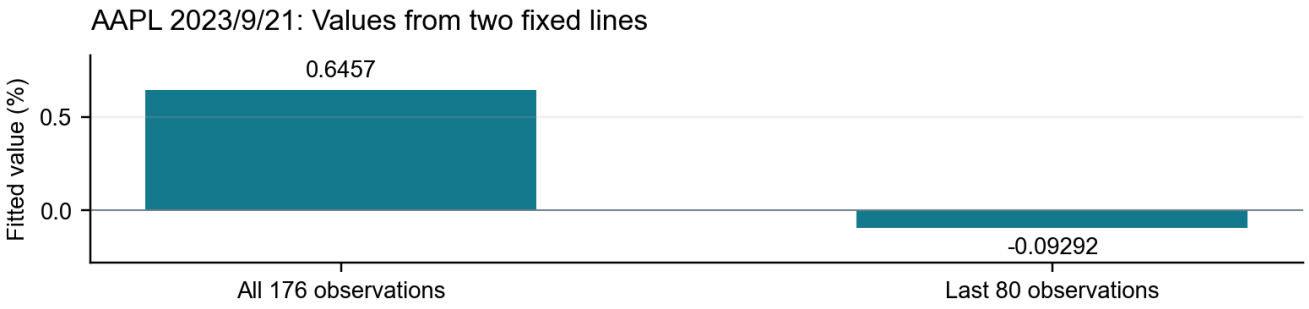
Change only the training observations used. Both OLS lines stay fixed, not refitted daily in 2024.

Each training observation has a date, its input x and the actual outcome y observed later. 176 and 80 are counts, not observation numbers 176 and 80: one line uses all 176 observations; the other uses only their last 80. Both have final input date 9/21 and complete outcomes through 9/29. They share a prediction target; these are not 80 independent trades.

Before running: Must the slope using only the latest 80 training observations be positive like the slope using all 176?

```
recent = train.tail(80)
recent_coef = np.linalg.lstsq(
    np.column_stack([np.ones(len(recent)), recent["x"]]), recent["y"], rcond=None)[0]
model_coefficients = {"All 176 observations": np.array([b0, b1]), "Last 80 observations": recent_coef}
pd.DataFrame([[name, len(rows), rows.index[0], round(100*coef[0], 6), round(coef[1], 8)]
    for name, rows, coef in [("All 176 observations", train, model_coefficients["All 176 observations"]),
                             ("Last 80 observations", recent, recent_coef)]],
    columns=["Candidate", "Training observations", "First input date", "Intercept (%)", "Slope"])
```

Candidate	Training observations	First input date	Intercept (%)	Slope
All 176 observations	176	2023-01-10	0.697743	0.05055818
Last 80 observations	80	2023-05-30	-0.193435	-0.0975928



At the same x≈-1.0299%, the line fitted to 176 observations gives 0.6457%; the line using 80 gives -0.0929%. This is a training observation, not a later validation result.

Changing the sample changes the slope from +0.05055818 to -0.09759280. More recent data alone do not establish greater accuracy; the next issue compares the same validation outcomes.

Discussion

For the same increase of 1 percentage point in x, in which direction does each line's prediction change? Explain using the two slopes.

Sources: [2], [3].

ISSUE 11

Using only 2023 validation, which OLS model should be locked?

Select one of the two fixed OLS candidates by mean squared error on 2023 validation, then keep it fixed for 2024.

Mean squared error (MSE) averages squared errors. Here i indexes the 12 validation periods. y and ŷ remain decimal returns; multiply their difference by 100 to obtain percentage points before squaring. MSE is in squared percentage points.

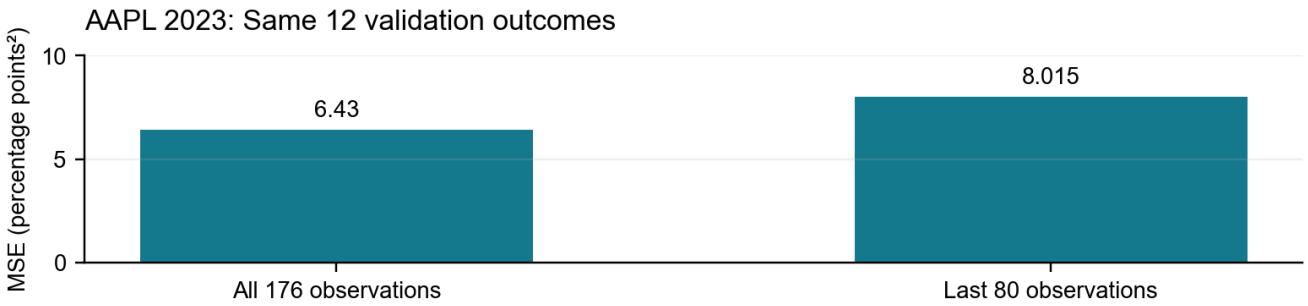
$$\mathrm{MSE} = \frac{1}{12}\sum_{i=1}^{12}[100(y_i - \hat{y}_i)]^2$$

Take one complete five-interval outcome every fifth observation: 12 decisions from 10/2 to 12/19, with the last outcome on 12/28. These are not the most favorable 12 validation periods, and no 2024 outcomes are used.

Before running: Must the latest 80 training observations give more accurate forecasts? Which candidate has lower MSE?

```
validation_rows, valid_predictions = [], {}
for name, coef in model_coefficients.items():
    pred = coef[0]+coef[1]*valid_cycles["x"]
    error_pp = 100*(valid_cycles["y"]-pred)
    valid_predictions[name] = pred
    validation_rows.append([name, len(pred), float((error_pp**2).mean())])
valid_scores = pd.DataFrame(validation_rows,
    columns=["Candidate", "Validation periods", "MSE (percentage points²)"])
selected = valid_scores.loc[valid_scores["MSE (percentage points²)"].idxmin(), "Candidate"]
locked_coef = model_coefficients[selected]
valid_scores.round(6)
```

Candidate	Validation periods	MSE (percentage points²)
All 176 observations	12	6.429727
Last 80 observations	12	8.014817



MSE using 176 training observations is about 6.429727, below 8.014817 using 80. The specified rule selects 176. Do not refit or reselect using 2024; later examples share this locked line.

Discussion

Does this page establish that OLS using 176 training observations is also more accurate than the training-mean forecast in 2024? Identify the candidates and outcome period compared.

Sources: [2], [4], [6].

ISSUE 12

How do we read six consecutive forecasts from the locked model?

Forecast every fifth trading observation. Check against the observed return from the next close to the five-interval endpoint.

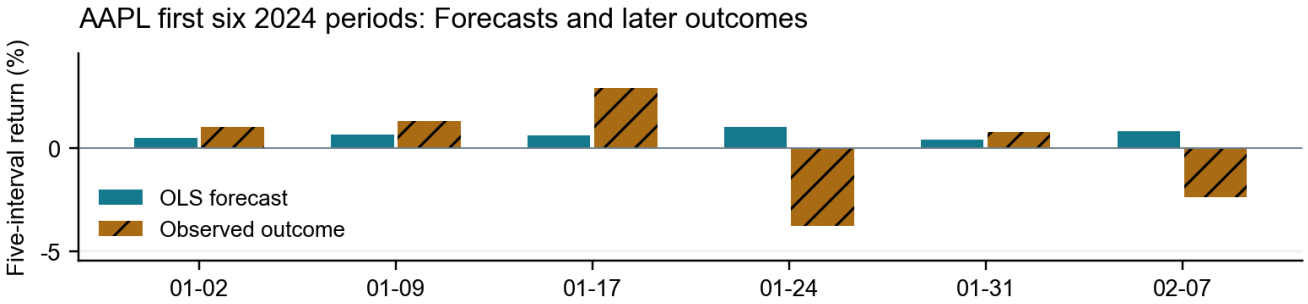
Use the selected line fitted to 176 training observations, fixing intercept 0.00697743 and slope 0.05055818. On 1/2 the past five-interval x≈-0.04111574. Substituting gives:

$$\hat{y}_{1/2} \approx 0.00697743 + 0.05055818(-0.04111574) \approx 0.00489870 = 0.4899\%$$

Before running: The 1/24 forecast exceeds 0. Must its outcome, known only on 2/1, also exceed 0?

```
fixed = locked_coef[0]+locked_coef[1]*test_cycles["x"]
preview = test_cycles.iloc[:6]
pd.DataFrame({"Decision date": preview.index, "Adjustment date": preview["Entry"].values,
    "Outcome date": preview["End"].values, "Prediction (%)": (100*fixed.iloc[:6]).round(4).values,
    "Outcome (%)": (100*preview["y"]).round(4).values})
```

Decision date	Adjustment date	Outcome date	Prediction (%)	Outcome (%)
2024-01-02	2024-01-03	2024-01-10	0.4899	1.0529
2024-01-09	2024-01-10	2024-01-18	0.6841	1.3105
2024-01-17	2024-01-18	2024-01-25	0.6306	2.937
2024-01-24	2024-01-25	2024-02-01	1.0249	-3.7647
2024-01-31	2024-02-01	2024-02-08	0.4352	0.7813
2024-02-07	2024-02-08	2024-02-15	0.8351	-2.3683



The 1/24 forecast is +1.0249%, but the observed 1/25 to 2/1 return is -3.7647%. The 2/7 forecast is +0.8351%, but 2/8 to 15 gives -2.3683%. Positive forecasts do not guarantee outcomes.

At the 1/9 decision, the first outcome ending 1/10 is still unknown. Use the locked model; do not add unfinished outcomes to training. These six periods are the consecutive beginning of all 24, not six separately chosen observations.

Discussion

Which two closes define the return estimated by 1.0249% on 1/24? Can it be called money already earned on 1/24?

Sources: [1], [2], [7].

ISSUE 13

What do baselines predict without using the current input?

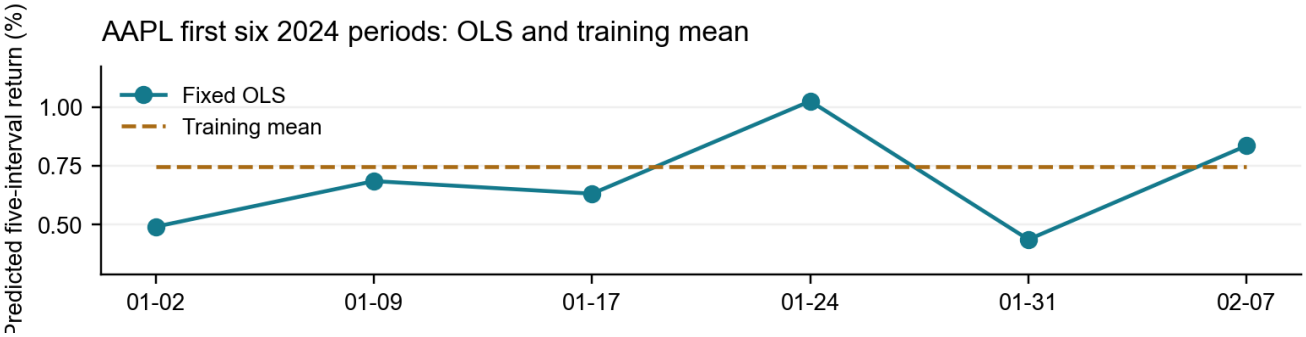
Use simple baselines unchanged by the current x to check whether OLS adds useful information.

The 0% forecast predicts 0 every period. The training-mean forecast predicts the mean of 176 training outcomes, +0.747204%, with outcomes through 2023/9/29. These are five-interval return forecasts, not prices or accounts; they use no 2024 outcomes.

Before running: Which two columns stay constant across the six forecasts even as x changes?

```
predictions = pd.DataFrame({"Fixed OLS": fixed,
    "0% forecast": 0.0, "Training-mean forecast": ybar})
shown = (100*predictions.iloc[:6]).round(4)
shown.insert(0, "Decision date", shown.index)
shown.columns = ["Decision date", "Fixed OLS (%)", "Forecast 0 (%)", "Training mean (%)"]
shown
```

Decision date	Fixed OLS (%)	Forecast 0 (%)	Training mean (%)
2024-01-02	0.4899	0.0	0.7472
2024-01-09	0.6841	0.0	0.7472
2024-01-17	0.6306	0.0	0.7472
2024-01-24	1.0249	0.0	0.7472
2024-01-31	0.4352	0.0	0.7472
2024-02-07	0.8351	0.0	0.7472



The training mean is 0.7472% each period, while OLS varies from 0.4352% to 1.0249%. The other baseline always forecasts 0%, a value of 0 rather than missing data.

OLS varies with the current x, but using an additional input does not itself establish greater accuracy. The next issue scores each forecast against the same outcomes.

Discussion

If you instead use the mean of the first six observed 2024 outcomes, is it still this page's training-mean forecast? Explain from the outcomes' source.

Sources: [2], [5].

ISSUE 14

What are the mean absolute and squared errors over six periods?

MAE measures the average absolute difference over six periods; MSE gives larger deviations more weight.

Mean absolute error (MAE) is in percentage points; mean squared error (MSE) is in squared percentage points. e remains outcome minus prediction in decimal returns. The table first lists OLS errors; the figures compare three forecasts. All use only the first six consecutive periods, not results for all 24.

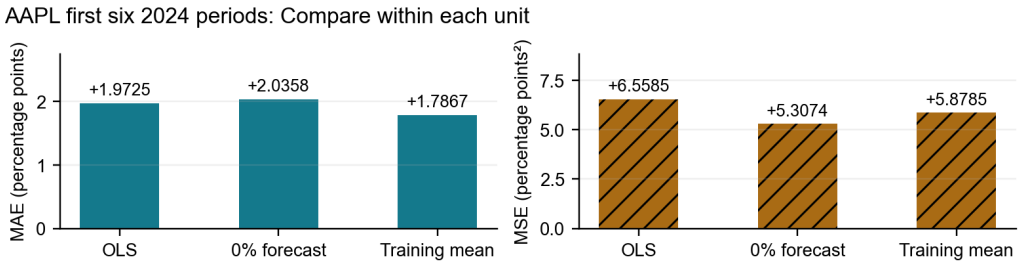
$$\mathrm{MAE} = \frac{1}{6}\sum|100e_i|, \qquad \mathrm{MSE} = \frac{1}{6}\sum(100e_i)^2$$

Before running: Is the square of the negative 1/24 error negative? Which squared error is largest?

```
six_error_pp = 100*(preview["y"]-fixed.iloc[:6])
six_scores = []
for name in predictions:
    error = 100*(preview["y"]-predictions[name].iloc[:6])
    six_scores.append([name, float(error.abs().mean()), float((error**2).mean())])
pd.DataFrame({"Decision date": preview.index, "OLS error (percentage points)": six_error_pp.values,
    "Absolute error (percentage points)": six_error_pp.abs().values,
    "Squared error (percentage points²)": six_error_pp.pow(2).values}).round(4)
```

Decision date	OLS error (percentage points)	Absolute error (percentage points)	Squared error (percentage points²)
2024-01-02	0.563	0.563	0.317
2024-01-09	0.6264	0.6264	0.3923
2024-01-17	2.3064	2.3064	5.3195
2024-01-24	-4.7896	4.7896	22.9404
2024-01-31	0.3461	0.3461	0.1198
2024-02-07	-3.2034	3.2034	10.2619

1/2: 1.0529%-0.4899%≈0.5630 percentage points; its square is about 0.3170. Apply the same calculation to the other five observations, checking each before averaging.



For six periods, OLS has MAE about 1.9725 and MSE about 6.5585. Compare methods within each side, not magnitudes across different units.

Discussion

Which contributes more to MSE, 1/24 or 1/17, and by approximately what factor? Square the displayed errors before comparing.

Sources: [2], [6].

ISSUE 15

How does the account continue through six holding periods?

Each period continues with the previous account value, rather than investing a fresh 1,000 USD each time.

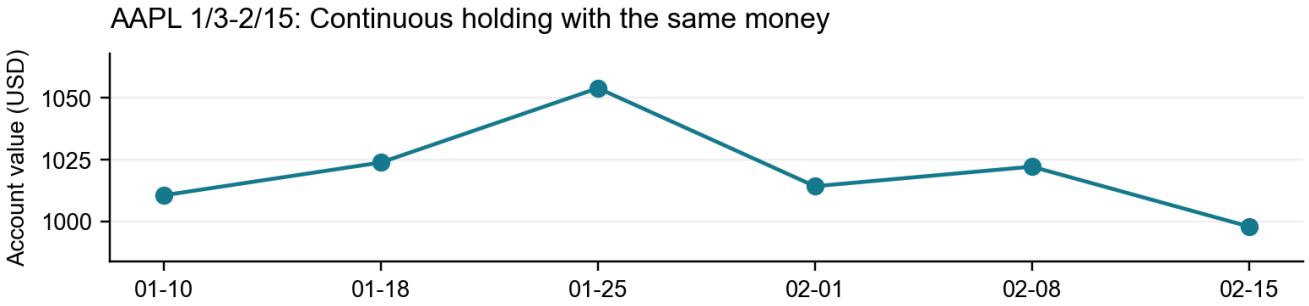
First use the no-fee rule: hold stock when the forecast exceeds 0%. All six OLS forecasts are positive, so buy on 1/3 and continue holding. Each endpoint matches the next period's start, leaving no gap or duplicated holding interval.

Before running: For the fourth period's decline of 3.7647%, should you multiply 1,000 or the previous 1,053.84 USD?

```
first_six_wealth = 1000*(1+preview["y"]).cumprod()
initial_shares = 1000/close.loc["2024-01-03"]
pd.DataFrame({"Period end": preview["End"].values,
    "Period stock return (%)": (100*preview["y"]).round(4).values,
    "Account value (USD)": first_six_wealth.round(2).values})
```

Period end	Period stock return (%)	Account value (USD)
2024-01-10	1.0529	1010.53
2024-01-18	1.3105	1023.77
2024-01-25	2.937	1053.84
2024-02-01	-3.7647	1014.17
2024-02-08	0.7813	1022.09
2024-02-15	-2.3683	997.88

$$1053.84(1 - 0.03764741) \approx 1014.17 \text{ USD}$$



Value is 1,010.53 on 1/10, rises to 1,053.84 on 1/25, and ends at 997.88 USD on 2/15. Even six positive forecasts can leave the account below initial capital.

The share count stays at 1,000÷184.25≈5.427408 shares. Multiply those shares by ending Close to check; the actual account is not updated using forecasts.

Discussion

Why does using 1,000×(1-3.7647%) for the fourth period not give the table's 1,014.17 USD?

Sources: [1], [7].

ISSUE 16

Must different prediction errors produce different trades?

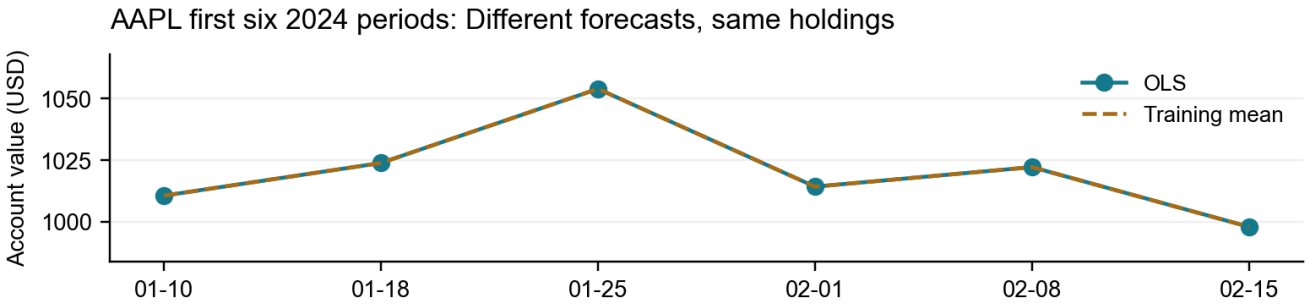
Forecast values differ, but a decision based only on whether they exceed 0% can produce identical trades.

Fix one decision rule here: hold stock if the forecast>0%, otherwise cash. A new buy uses all cash; continuing to hold retains the shares. No fees are charged. Update the actual account with realized stock outcomes, not forecast returns.

Before running: OLS and the training mean both exceed 0%. Must differing forecast values create different accounts?

```
requirements = predictions > 0
gross_period_wealth = 1000*(1+requirements.mul(test_cycles["y"], axis=0)).cumprod()
pd.DataFrame({"Forecast method": predictions.columns,
    "Periods requiring stock": requirements.sum().values,
    "6/26 account (USD)": gross_period_wealth.iloc[-1].round(2).values})
```

Forecast method	Periods requiring stock	6/26 account (USD)
Fixed OLS	24	1157.39
0% forecast	0	1000.0
Training-mean forecast	24	1157.39



The OLS and training-mean lines coincide: both require stock for six periods and reach 997.88 USD on 2/15. The 0% forecast fails the strict >0% condition and stays in cash at 1,000 USD.

Across all 24 periods, OLS forecasts range about 0.3670%-1.3143%, and the training mean stays at 0.7472%; all exceed 0. Both buy on 1/3 and continue holding, reaching 1,157.39 USD on 6/26, not buying 24 times each.

Discussion

Under this rule, does changing a positive forecast from 0.4% to 0.7% change the trades? What threshold crossing changes the holding requirement?

Sources: [2], [6].

ISSUE 17

How do forecasts above 0 or 0.8% change purchase dates?

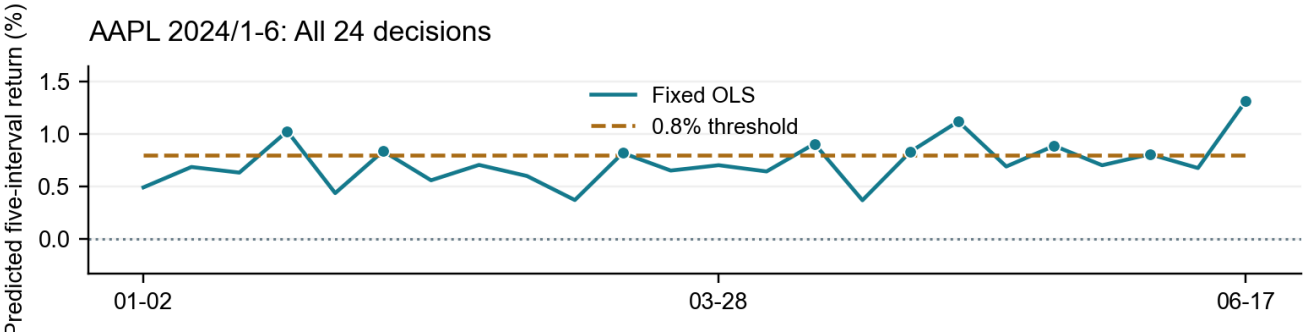
Use the same OLS forecasts but hold stock only above 0.8%. Keep the model, scheduled decision dates and next-close execution assumption unchanged.

0.8% is a preset classroom threshold, not an optimum selected from 2024 or an estimated trading cost. Equality still requires cash.

Before running: Of the 1/24 and 1/31 forecasts, which requires stock and which a return to cash?

```
threshold_required = fixed > 0.008
threshold_wealth = 1000*(1+test_cycles["y"]*threshold_required).cumprod()
prior_required = threshold_required.shift(1, fill_value=False)
threshold_buys = threshold_required & ~prior_required
threshold_sells = ~threshold_required & prior_required
pd.DataFrame({"Decision date": preview.index,
    "Prediction (%)": (100*fixed.iloc[:6]).round(4).values,
    "Next close": preview["Entry"].values,
    "0.8% rule": threshold_required.iloc[:6].map({True: "Hold stock", False: "Cash"}).values})
```

Decision date	Prediction (%)	Next close	0.8% rule
2024-01-02	0.4899	2024-01-03	Cash
2024-01-09	0.6841	2024-01-10	Cash
2024-01-17	0.6306	2024-01-18	Cash
2024-01-24	1.0249	2024-01-25	Hold stock
2024-01-31	0.4352	2024-02-01	Cash
2024-02-07	0.8351	2024-02-08	Hold stock



Filled points mark the 9 periods with forecasts>0.8%, not 9 new purchases. The 1/24 forecast is 1.0249%, leading to a 1/25 purchase; 0.4352% on 1/31 leads to a 2/1 sale. See the first six outcomes and Issue 22 for observed returns; forecasts are not returns already earned.

All 24 OLS forecasts>0%, so buy on 1/3 and continue holding; value is 1,157.39 USD on 6/26. With >0.8%: hold for 9 periods, with 8 buys and 7 sells. Each new buy uses 100% of cash; continued holdings are not rebalanced. Ending value without fees is 1019.89 USD, still in stock and only valued. The higher threshold gives lower ending value here.

Discussion

If 0.8% becomes 1.0%, do the holding requirements on 1/24 and 2/7 change? Compare the forecasts without refitting.

Sources: [2], [7].

ISSUE 18

How do new buys using 100% or 50% of cash change the account?

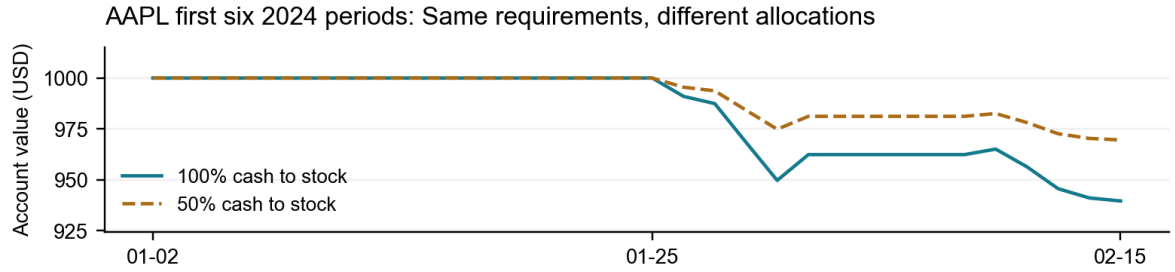
Keep the >0.8% rule. On each new buy, invest 100% or 50% of available cash after fees; retain the rest as cash.

Here fee=0. Initial cash of 1,000 USD buys 1,000 USD of stock and leaves 0 cash, or buys 500 USD and leaves 500 cash. required is the stockholding requirement; fraction is the cash fraction invested on a new buy. Continued holding keeps shares, without restoring 50%. Sell all only when cash is required.

Before running: If a new buy uses 50% of cash, must the stock weight still be 50% when holding through 5/6?

```
study = close.loc["2024-01-02":test_cycles["End"].iloc[-1]]
def account(required, fraction=1, fee=0):
    orders = dict(zip(test_cycles["Entry"], required))
    cash, shares, rows = 1000.0, 0.0, []
    for day, price in study.items():
        before = cash+shares*price
        charged, action = 0.0, "Continue holding" if shares>0 else "Cash"
        if day in orders and bool(orders[day]) != (shares>0):
            charged = fee
            if orders[day]:
                cash -= fee
                amount = fraction*cash
                shares, cash = amount/price, cash-amount
                action = "Buy"
            else:
                cash, shares = cash+shares*price-fee, 0.0
                action = "Sell"
        rows.append([day, shares, cash, before, cash+shares*price, charged, action])
    return pd.DataFrame(rows, columns=["date", "shares", "cash", "before",
        "wealth", "fee", "action"]).set_index("date")
full_gross = account(threshold_required)
half_gross = account(threshold_required, fraction=0.5)
pd.DataFrame({"Cash used on a new buy": ["100% to stock", "50% to stock"], "6/26 total value (USD)":
    [round(full_gross.wealth.iloc[-1], 2), round(half_gross.wealth.iloc[-1], 2)]})
```

Cash used on a new buy	6/26 total value (USD)
100% to stock	1019.89
50% to stock	1011.52



On 1/25 both accounts have 1,000 USD: 100% buys 1,000 of stock and leaves 0 cash; 50% buys 500 and leaves 500 cash. Values on 2/15 are 939.56 and 969.56 USD respectively, without fees.

Discussion

After buying stock with 50% of cash on 4/29, its weight on 5/6 is about 51.1557%. Does this rule require selling some to restore 50%?

Sources: [1], [7].

ISSUE 19

Does a positive ending-return forecast imply a rise every day?

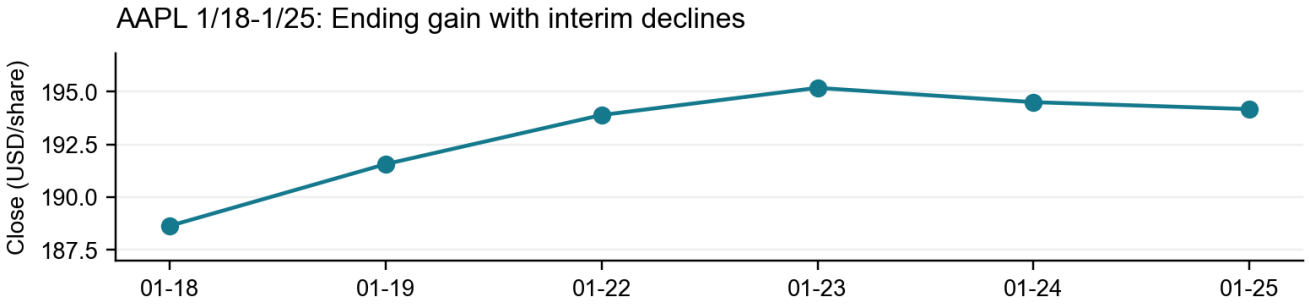
OLS forecasts only the five-interval ending return, not the order of daily price changes within it.

The 1/17 forecast is +0.6306%. Buying at the 1/18 close gives an observed +2.9370% by 1/25. Examine the same period's six observed prices without changing the stock or selecting a different outcome.

Before running: If 1/25 is above 1/18, must both 1/24 and 1/25 exceed the previous day?

```
positive_block = close.loc["2024-01-18":"2024-01-25"]
block_daily = positive_block.pct_change()
block_return = positive_block.iloc[-1]/positive_block.iloc[0]-1
pd.DataFrame({"Observation date": positive_block.index,
    "Close (USD/share)": positive_block.round(2).values,
    "Change from previous (%)": ["Purchase start" if pd.isna(r) else f"{100*r:.4f}"
        for r in block_daily]})
```

Observation date	Close (USD/share)	Change from previous (%)
2024-01-18	188.63	Purchase start
2024-01-19	191.56	1.5533
2024-01-22	193.89	1.2163
2024-01-23	195.18	0.6653
2024-01-24	194.5	-0.3484
2024-01-25	194.17	-0.1697



The price is 188.63 on 1/18, rises to 195.18 on 1/23, then falls to 194.50 and 194.17. The endpoint remains above purchase price; the model provides no daily path.

$$\frac{194.17}{188.63} - 1 \approx 2.9370\%$$

The same ending return can accompany different interim fluctuations. Exiting at a loss or profit requires separate trigger and execution rules. The ending forecast does not guarantee daily prices.

Discussion

Use the starting and ending prices of the two declines in the table to refute: "This period earned a profit, so the price rose every day."

Sources: [1], [2].

ISSUE 20

After an early stop-loss exit, is profit/loss still the original five-interval outcome?

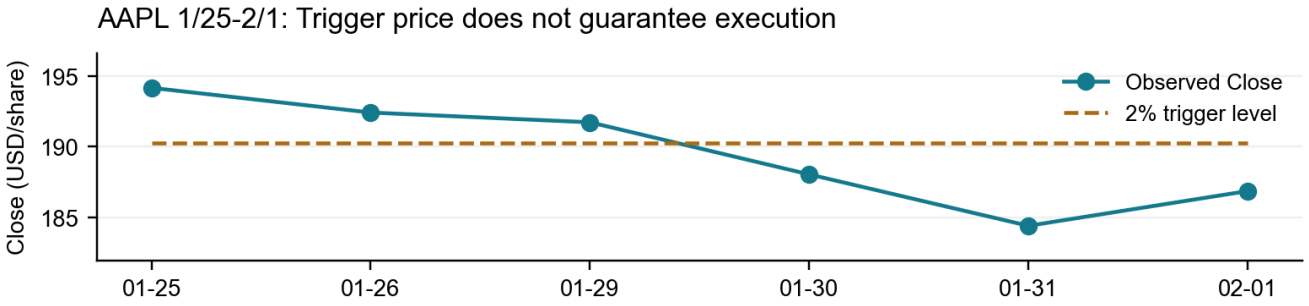
Selling early changes the actual holding period, not the original five-interval outcome the model forecasts.

Start a separate single-trade example with 1,000 USD: forecast +1.0249% on 1/24 and buy on 1/25. If a close is at least 2% below purchase price, sell at the next close, without fees. This is not a brokerage stop order and does not change the preceding 24-period account.

Before running: Does triggering a 2% decline guarantee a loss of only 20 USD? Could the next close be lower?

```
negative_block = close.loc["2024-01-25":"2024-02-01"]
stop_level = 0.98*negative_block.iloc[0]
stop_hit = negative_block[negative_block <= stop_level].index[0]
stop_exit = negative_block.index[negative_block.index.get_loc(stop_hit)+1]
stop_cash = 1000*negative_block.loc[stop_exit]/negative_block.iloc[0]
planned_negative = 1000*negative_block.iloc[-1]/negative_block.iloc[0]
dates = [negative_block.index[0], stop_hit, stop_exit, negative_block.index[-1]]
pd.DataFrame({"Observation date": dates, "Close (USD/share)": negative_block.loc[dates].round(2).values,
    "Event": ["Buy", "Trigger", "Sell", "Original outcome end"]})
```

Observation date	Close (USD/share)	Event
2024-01-25	194.17	Buy
2024-01-30	188.04	Trigger
2024-01-31	184.4	Sell
2024-02-01	186.86	Original outcome end



Purchase price is 194.17; the 2% condition is about 190.29. The 1/30 close of 188.04 triggers it, but the assumed sale is at 184.40 on 1/31, giving a decline greater than 2%.

$$1000\frac{184.40}{194.17} \approx 949.68 \text{ USD}$$

At the common 2/1 endpoint, the early exit holds 949.68 USD cash, versus stock worth 962.35 without exiting. Selling early leaves 12.67 USD less here. The original outcome remains -3.7647% over 1/25→2/1, not the shorter realized return.

Discussion

The actual 1/31 exit return is about -5.0317%. Should it replace original y when scoring the five-interval forecast made on 1/24?

Sources: [1], [8].

ISSUE 21

After early profit-taking, how does the return differ from the original outcome?

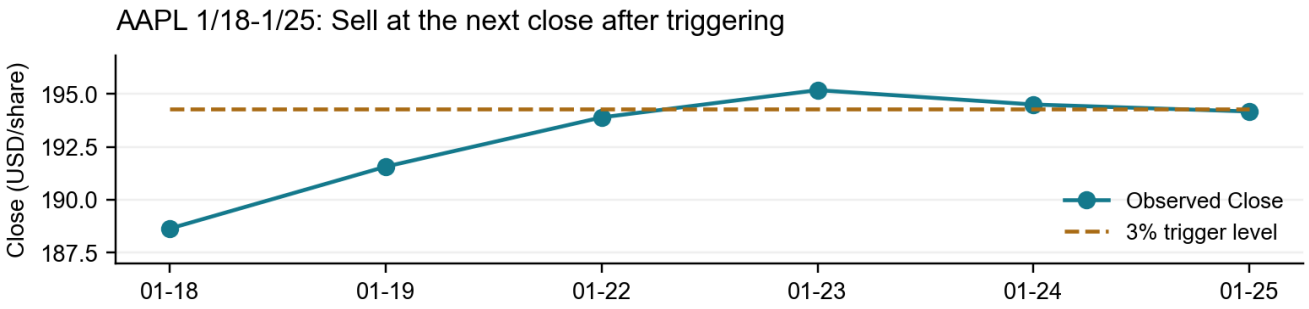
When a close reaches a 3% gain, sell at the next close. Reaching the trigger does not mean the realized gain is exactly 3%.

Start a separate single-trade example with 1,000 USD: forecast +0.6306% on 1/17 and buy on 1/18 under the >0% rule, not the >0.8% account. Hold cash after selling early and compare at the common 1/25 endpoint, without fees.

Before running: Can the Close of 195.18 that triggers the rule on 1/23 be used as the assumed sale price on 1/24?

```
profit_level = 1.03*positive_block.iloc[0]
profit_hit = positive_block[positive_block >= profit_level].index[0]
profit_exit = positive_block.index[positive_block.index.get_loc(profit_hit)+1]
profit_cash = 1000*positive_block.loc[profit_exit]/positive_block.iloc[0]
planned_positive = 1000*(1+block_return)
dates = [positive_block.index[0], profit_hit, profit_exit, positive_block.index[-1]]
pd.DataFrame({"Observation date": dates, "Close (USD/share)": positive_block.loc[dates].round(2).values,
    "Event": ["Buy", "Trigger", "Sell", "Original outcome end"]})
```

Observation date	Close (USD/share)	Event
2024-01-18	188.63	Buy
2024-01-23	195.18	Trigger
2024-01-24	194.5	Sell
2024-01-25	194.17	Original outcome end



The 3% condition is about 194.29; 195.18 on 1/23 triggers it, followed by an assumed sale at the 1/24 close of 194.50. Do not substitute that higher observed Close for the specified next Close.

$$\frac{194.50}{188.63} - 1 \approx 3.1119\%$$

At the common 1/25 endpoint, early sale leaves 1031.12 USD cash, versus 1029.37 when holding. The extra 1.75 USD from this single trade does not establish that profit-taking always increases profits.

Discussion

The exit return is 3.1119%, versus the original five-interval outcome of 2.9370%. Why do they differ, and which scores the 1/17 forecast?

Sources: [1], [8].

ISSUE 22

Across all 24 periods, must more accurate forecasts earn more?

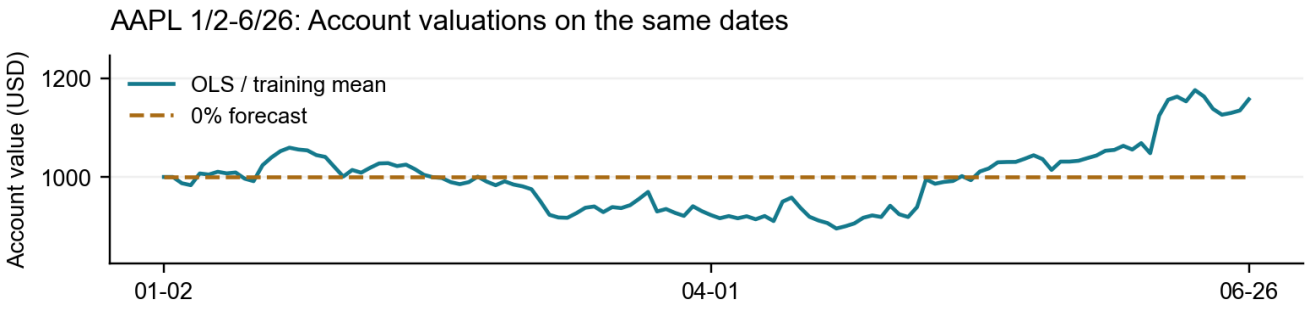
Use all 24 periods, not one observation or the first six. Smaller forecast errors need not produce a higher account value.

Score all three forecasts against the same 24 stock-return outcomes. Each account holds stock when its own forecast>0%, without fees. MAE uses percentage points, MSE squared percentage points, and accounts USD; do not combine them into one ranking.

Before running: If the training mean has smaller errors, must its ending account under the >0% rule exceed OLS?

```
full_scores, prediction_accounts = [], {}
for name in predictions:
    error = 100*(test_cycles["y"]-predictions[name])
    ledger = account(predictions[name] > 0)
    prediction_accounts[name] = ledger
    full_scores.append([name, float(error.abs().mean()), float((error**2).mean()),
        float(ledger.wealth.iloc[-1])])
pd.DataFrame(full_scores, columns=["Forecast method", "MAE (percentage points)",
    "MSE (percentage points²)", "6/26 total value (USD)"]).round(4)
```

Forecast method	MAE (percentage points)	MSE (percentage points²)	6/26 total value (USD)
Fixed OLS	2.4103	8.3406	1157.3948
0% forecast	2.387	8.4617	1000.0
Training-mean forecast	2.3247	8.0473	1157.3948



OLS and the training mean both buy on 1/3 and continue holding, reaching 1,157.39 USD on 6/26. The 0% forecast holds 1,000 USD cash throughout, but its errors are still scored against the 24 stock returns, not cash returns.

The training mean has MAE 2.3247 and MSE 8.0473, below OLS's 2.4103 and 8.3406, yet identical holding requirements. Compared with the 0% forecast, OLS has lower MSE but higher MAE. State which metric defines "more accurate."

Discussion

Why do both accounts equal 1,157.39 USD when the training mean has lower MSE than OLS? Explain using forecast signs and actual trades.

Sources: [1], [2], [6].

ISSUE 23

Do repeated stockholding requirements need a sale and repurchase?

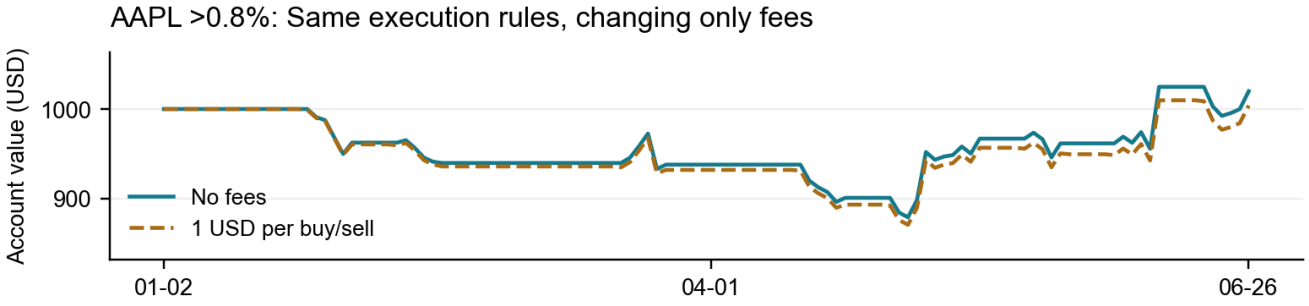
Another forecast requiring stock does not mean another purchase. Charge fees only on actual buy or sell dates.

Reuse account with fee set to 1 USD. Deduct the fee before investing remaining cash in a new purchase. An unchanged holding requirement retains the shares, without selling, repurchasing or charging a fee. The endpoint values stock without selling.

Before running: Both 4/29 and 5/6 require stock. Should the share quantity or trading fee change on 5/6?

```
full_net = account(fixed > 0, fee=1)
threshold_net = account(threshold_required, fee=1)
dates = ["2024-04-29", "2024-05-06", "2024-05-13"]
shown = threshold_net.loc[dates, ["action", "shares", "fee", "wealth"]].copy()
shown["shares"], shown["wealth"] = shown["shares"].round(6), shown["wealth"].round(2)
shown.insert(0, "Observation date", shown.index)
shown.columns = ["Observation date", "Action", "Shares", "Fee (USD)", "Total value (USD)"]
shown
```

Observation date	Action	Shares	Fee (USD)	Total value (USD)
2024-04-29	Buy	5.140664	1.0	891.91
2024-05-06	Continue holding	5.140664	0.0	934.11
2024-05-13	Sell	0.0	1.0	956.6



The full period has 8 buys and 7 sells, with nominal fees of 15 USD. On 6/26, value is 1019.89 without fees and 1003.91 USD with fees. Fees reduce later investable amounts, so the ending-value difference need not equal 15.

Keep the same shares on 5/6, without a trade or fee; sell all on 5/13. The other >0% rule holds stock for all 24 periods, charging only 1 USD for the 1/3 purchase. Stock remains on 6/26, so no sale fee is charged.

Discussion

If every stockholding requirement incurs 1 USD, how much extra is charged on 5/6? Does this count actual trades or decisions?

Sources: [1], [7].

ISSUE 24

Do buys using 100% or 50% of cash after fees still profit?

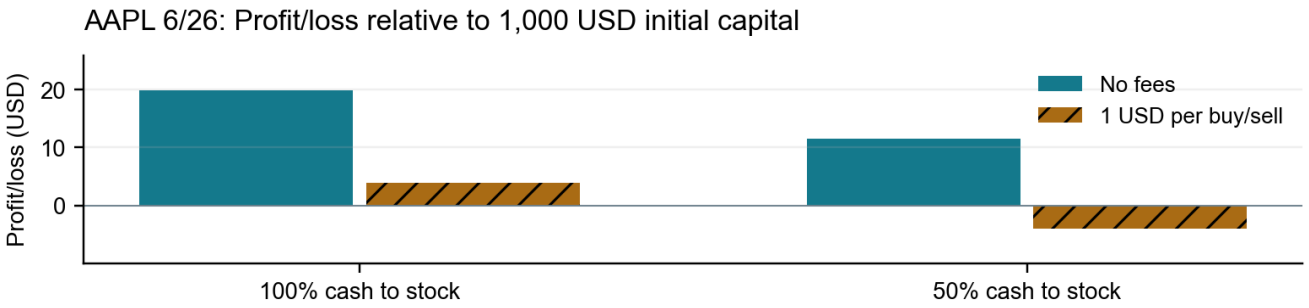
Keep the >0.8% rule and the same 15 trades. On each new buy, deduct 1 USD first, then invest 100% or 50% of available cash in stock.

Initially deduct 1 USD from 1,000, leaving 999 USD. Investing 100% buys 999 of stock and leaves 0 cash; 50% buys 499.50 and leaves 499.50 cash. Later new buys also use available cash after fees. Continued holding is not rebalanced to 50%.

Before running: If the stock investment is halved, does the fixed 1 USD trading fee automatically halve too?

```
half_net = account(threshold_required, fraction=0.5, fee=1)
allocation_results = pd.DataFrame({"Cash used after fees": ["100% to stock", "50% to stock"],
    "Without fees (USD)": [full_gross.wealth.iloc[-1], half_gross.wealth.iloc[-1]],
    "After fees (USD)": [threshold_net.wealth.iloc[-1], half_net.wealth.iloc[-1]],
    "Nominal fees (USD)": [threshold_net.fee.sum(), half_net.fee.sum()]})
allocation_results.round(2)
```

Cash used after fees	Without fees (USD)	After fees (USD)	Nominal fees (USD)
100% to stock	1019.89	1003.91	15.0
50% to stock	1011.52	996.03	15.0



Without fees, using 100% / 50% of cash to buy stock earns 19.89 / 11.52 USD. After fees, the accounts earn 3.91 / lose 3.97 USD. The model and trading dates are the same; stock purchase amounts differ.

After fees, using 100% of cash gives ending value 1,003.91 USD; 50% gives 996.03 USD. Each new buy uses then-available cash, not a reset to 1,000. Fees affect later shares; simply subtracting 15 from the no-fee ending value is incorrect.

Discussion

Using 50% of cash to buy stock profits without fees but loses after fees. Why does halving the stock purchase amount not halve the fixed fees?

Sources: [1], [7].

ISSUE 25

When input exceeds the training range, does a higher forecast justify investing more?

A line can still produce a number; that does not mean this input has been tested by the training data.

The past five-interval x values of the 176 training observations range only from -8.9590% to +7.1800%. Among scheduled 2024 decision dates, +8.3166% on 5/3 and +12.1945% on 6/17 exceed the upper bound; only these two do. The fixed model is not re-estimated.

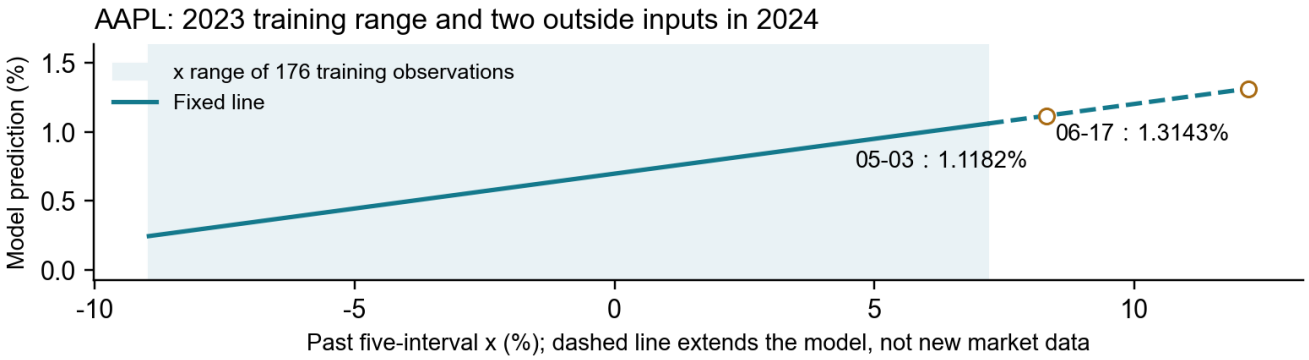
x and ŷ still use decimal returns. The line has intercept 0.00697743 and slope 0.05055818. On 6/17, convert 12.1945% to 0.121945 and substitute:

$$\hat{y}_{6/17} \approx 0.00697743 + 0.05055818(0.121945) \approx 1.3143\%$$

Before running: Both forecasts are >0.8%. Must each lead to a purchase at the next close?

```
train_min, train_max = train["x"].min(), train["x"].max()
outside = test_cycles.loc[~test_cycles["x"].between(train_min, train_max)]
pd.DataFrame({"Decision date": outside.index,
    "Past five intervals (%)": (100*outside["x"]).round(4).values,
    "Prediction (%)": (100*fixed.loc[outside.index]).round(4).values,
    "Next-close action": full_gross.loc[outside["Entry"], "action"].values,
    "Outcome date": outside["End"].values})
```

Decision date	Past five intervals (%)	Prediction (%)	Next-close action	Outcome date
2024-05-03	8.3166	1.1182	Continue holding	2024-05-13
2024-06-17	12.1945	1.3143	Buy	2024-06-26



The 5/3 forecast of 1.1182% leads to continued holding on 5/6; 1.3143% on 6/17 leads to a new purchase on 6/18. Higher values reflect the positive slope extending to unseen x (extrapolation), not a rise probability or smaller error. Predictions inside the range are not guaranteed correct; predictions outside need not be wrong.

A new rule forbidding new purchases outside the range must be specified first, then compared on subsequent data; its effect is not tested here. All x values in the 12 validation periods of 2023 were within the original range, so neither larger input was tested.

Discussion

The 6/17 forecast is 1.3143%, higher than on 5/3. Does this alone justify changing the original 100% / 50% cash rule to borrowing for a larger position? Explain using the training range and prediction units.

Sources: [2], [9].

References and Reproduction Conditions

Retained Price Data and Model Assumptions

[1] yfinance: Historical price fields

[2] James et al.: ISL chapter 3, equations 3.3-3.4

[3] NumPy: linalg.lstsq

[4] scikit-learn: Time-ordered cross-validation

[5] scikit-learn: Data preprocessing and information leakage

[6] scikit-learn: Mean squared error

[7] pandas: shift

[8] Investor.gov: Execution limits of stop-loss and trailing-stop orders

[9] NIST: Prediction outside the data range and validation

AAPL: 374 prices over 2023/1/3-2024/6/28, using the provider response retained on 2026/9/18. No prices were downloaded or rewritten in this revision. Close is in USD/share, not total return including dividends.

The input is an observed five-interval return; the outcome covers five intervals after adjustment at the next Close. After 2023 validation selects OLS fitted to 176 training observations, make one decision every five trading observations in 2024. The 24 complete intervals share valuation through 6/26, without an extra terminal trade.

Close execution, fractional shares, preset 0%/0.8% thresholds, partial allocation, exits and 1 USD per buy or sell are classroom assumptions. Cash earns no interest; dividends, taxes and actual bid-ask spreads are excluded. Forecasts do not guarantee profit; stock still held is not bought again.

Displayed code runs in order in a fresh Python process. Python 3.12.14; numpy 2.2.6; pandas 2.2.3; matplotlib 3.10.8; yfinance 1.2.0. The instructor retains full data and checks; student materials exclude answers.